

A1 · Answer Key and Demo Script

CS425 · Speaking the Application Layer

INSTRUCTOR COPY

Graded pass/fail. Every transcript below is real output captured from `onyx.boisestate.edu` while this was written, so you can run any of it at the podium and get the same shape. Numbers will differ; the shapes will not.

WHAT IS ON THE BOX

THING	MEASURED ON ONYX
OS	Red Hat Enterprise Linux 9.8 (Plow), kernel 5.14
dig	DiG 9.16.23-RH
curl	7.76.1, OpenSSL 3.5.5, nghttp2
nc	nmap-ncat 7.92 at <code>/usr/bin/ncat</code> , not BSD netcat
ping	iputils 20210202
resolvers	10.50.18.111 and 10.50.18.112
outbound 53 elsewhere	blocked , which is why two dig switches fail

ROUND 1 · GET ON THE BOX

PASS BAR: a release string, a head count, and four version lines. The only thing to check is that `nc` says **Ncat**; a group that wrote "netcat" copied it from a neighbor on a Mac.

ROUND 2 · DIG

Reading the answer

```
$ dig boisestate.edu
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 59539
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
boisestate.edu.      300    IN      A       132.178.96.59
;; Query time: 1 msec
;; SERVER: 10.50.18.111#53(10.50.18.111)
```

SAY: four things to point at. **STATUS** is the verdict. **ANSWER: 1** is how many records came back. **300** is the TTL. **SERVER** is who answered, and it is a campus box, not the internet.

The TTL, and why the original version of this exercise was wrong

```
$ dig +noall +answer boisestate.edu # five times
boisestate.edu.      300    IN      A       132.178.96.59    ← 300 every single time
```

```
$ dig +noall +answer example.com ; sleep 3 ; dig +noall +answer example.com
example.com.         218    IN      A       172.66.147.243
example.com.         241    IN      A       172.66.147.243    ← it went UP
```

SAY: `boisestate.edu` never moves because the campus resolver is **AUTHORITATIVE** for that zone: it is not caching the answer, it *is* the answer. `example.com` moves because it really is cached, and it sometimes moves **UP** because `resolv.conf` lists two resolvers and you are getting whichever answers first, with caches of different ages. A group that notices the increase and is bothered by it has understood caching better than one that just wrote "it counts down".

The two switches that fail, and the one that lies

```
$ dig @1.1.1.1 boisestate.edu
;; connection timed out; no servers could be reached

$ dig +trace boisestate.edu
;; Received 239 bytes from 10.50.18.111#53(10.50.18.111) in 19 ms ← and then it stops

$ dig nosuchthing.boisestate.edu
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 8848
;; flags: qr rd ra; QUERY: 1, ANSWER: 0

$ dig +short nosuchthing.boisestate.edu
(nothing at all, and it exits 0)
```

SAY: the first two fail for **THE SAME REASON:** both need to send a query directly to a DNS server out on the internet, and outbound port 53 is blocked. That is a middlebox making a policy decision, and in A4 they meet one that answers *wrongly* instead of not at all, which is worse. **THEN THE TRAP:** +short prints an empty line for a name that does not exist. The empty line is a real answer, badly displayed. This is A4 station 4, exactly.

The nuance worth having ready

Someone will run `dig +short boisestate.edu AAAA` and get nothing, and conclude the name does not exist. It does. The full output says **status: NOERROR** with **ANSWER: 0: the name exists, that record type does not**. Two different causes, one identical empty line, and only the long form tells them apart. If a group finds this on their own, put it on the board.

PASS BAR: the status field, answer and TTL are written down, and the group noticed that +short printed nothing for the bad name.

Running the room

- **Say why first.** The seven round version of this activity died in round 2 because nobody knew what it was for. Two minutes at the board before the paper goes out: every URL starts with a DNS lookup, today you send one by hand and read the answer, two rounds, the only way to fail is a blank box.
- **Show, do not explain.** Put `dig boisestate.edu` on the projector and point at the four things the sheet asks for. Finding them again is the group's job.
- **Round 1 is fifteen minutes**, most of it passwords. Nobody has keys yet; that is A2.
- **If time is short**, cut +tcp and +stats. Keep the two that fail and the +short trap; A4 depends on both.
- **The copy tell.** A group that wrote "asks Cloudflare" for @1.1.1.1 instead of "timed out" did not run it.